



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**DYNAMIC ONLINE JOB PORTAL AND RECRUITMENT
MANAGEMENT SYSTEM
A CAPSTONE PROJECT REPORT**

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA4305 Internet Programming

to the award of the degree of
BACHELOR OF ENGINEERING
IN
Computer Science and Engineering

Submitted by

S. Shashi Vardhan (192311155)

P. Praveen (192372105)

G. Naveen (192324110)

N. lokabandhava(192324167)

Under the Supervision of

Dr. D. Sudhagar

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER-2025

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Dynamic Online Job Portal and Recruitment Management System" has been carried out by S. Shashi Vardhan Reddy (192311155) under the supervision of Dr. D. Sudhagar and is submitted in partial fulfilment of the requirements for the current semester of the B.E. Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. Marie Claude .N

Professor

Department of Neural Networks
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. D. Sudhagar

Professor

Department of Quantum Intelligence

DECLARATION

We, S. Shashi Vardhan Reddy (192311155) of the Computer Science and Engineering Department, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Dynamic Online Job Portal and Recruitment Management System" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 03-09-2026

Signature of the Students with Names

S. Shashi Vardhan Reddy (192311155)

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Reg. No.	Module & Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. %
S.Shashi Vardhan Reddy 192311155	Module 1: Front-end & Job Search — job listing UI, search & filter interface	Developed HTML/CSS/JavaScript pages, job search and filter interface, responsive layout	Tested front-end forms, navigation, and cross-device responsiveness	Solution Design, Tools & Technologies, Appendices	25%
P. Praveen 192372105	Module 2: Backend & Employer Job Posting — job posting, employer dashboard, database	Developed PHP backend, MySQL schema, employer jobposting workflow and MVC integration	Tested backend request handling, database queries, and system integration	Introduction, Solution Design, Reflection, Conclusion	25%
G. Naveen 192324110	Module 3: Candidate Application & Recruitment Workflow — applications, shortlisting, status tracking	Developed the application submission, shortlisting/approval workflow, and statustracking module	Tested applicationsubmission flows and status-update notifications	Problem Identification, Results and Recommendations	25%
N. Lokabandhava 192324167	Module 4: Admin Panel & Reports — admin dashboard, user/job moderation, analytics & reports	Developed the admin panel, moderation controls, and the reporting/analytics module (recruitment statistics, exportable reports)	Tested admin authentication, moderation actions, and report-generation accuracy	Engineering Standards, References, Appendices	25%

Total: 100%

ABSTRACT

The project titled "Dynamic Online Job Portal and Recruitment Management System" is aimed at creating a centralized, cloud-hosted web application that connects job seekers with employers and streamlines the end-to-end recruitment lifecycle. Traditional recruitment channels — newspaper listings, informal referrals, and fragmented job boards — often suffer from limited visibility, slow communication, and a lack of structured tracking between

the moment a job is posted and the moment a candidate is hired. This project addresses these gaps by developing a platform where employers can post job openings with detailed requirements, candidates can search and filter listings, submit applications with resumes, and track the status of every application in real time.

The platform emphasizes transparency and efficiency by automating key stages of the recruitment funnel: job posting, application submission, shortlisting, interview scheduling notifications, and final status updates. Secure user authentication distinguishes between candidate, employer, and administrator roles, each with a tailored dashboard. Cloud computing technology underpins the system to ensure scalability, reliability, and real-time synchronization of job listings and application data across devices.

A dedicated Admin Panel & Reports module gives platform administrators the ability to moderate job postings and user accounts, monitor platform activity, and generate analytical reports — including the volume of postings by category, application-to-hire conversion rates, and employer activity summaries — supporting data-driven oversight of the recruitment ecosystem. The backend integrates a responsive front-end interface, a PHP-based backend, and a MySQL cloud-hosted database that stores user profiles, job listings, applications, and transaction/audit histories, with validation and conflict-resolution mechanisms to prevent duplicate applications or unauthorized actions.

Overall, the platform aims to reduce the friction and inefficiency of traditional recruitment processes, giving job seekers wider and fairer access to opportunities while giving employers a structured, auditable pipeline for sourcing and evaluating talent.

TABLE OF CONTENTS

BONAFIDE CERTIFICATE	2
DECLARATION	3
INDIVIDUAL CONTRIBUTION STATEMENT	4
ABSTRACT	5
TABLE OF CONTENTS	6
LIST OF FIGURES	8
LIST OF TABLES	9
LIST OF ABBREVIATIONS	10
CHAPTER 1: INTRODUCTION	11
1.1 Background Information	11
1.2 Need for the Project	11
1.3 Project Aim	11
1.4 Project Objectives	11
1.5 Scope	12
1.6 Expected Engineering Outcome	12
System	12
Product	12
Process	12
Prototype	12
Model	12
CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	13
2.1 Review Method	13
2.2 Review of Existing Work	13
2.3 Comparative Analysis	13
2.4 Research / Engineering Gap	14
2.5 Proposed Contribution	14
CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	15

3.1 Requirements	15
Stakeholder / User Requirements	15
Functional & Non-Functional Requirements	15
3.2 Constraints & Applicable Standards	15
3.3 Design Specifications	16
3.4 Alternative Solutions & Evaluation	16
Alternative 1: Informal / manual recruitment (referrals, classifieds)	16
Alternative 2: Existing commercial job boards / enterprise ATS	16
Alternative 3: Dynamic Online Job Portal (proposed)	17
3.5 Selected Design & Detailed Design	17
Justification for Final Design Selection	17
Architecture / Flowchart	17
3.6 Trade-off Analysis	17
3.7 Sustainability, Ethics, Safety, Risk & Security	17
Risk Register	17
CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	18
4.1 Development Approach & Resources	18
4.2 Software Implementation & Tools Used	18
4.3 Module-Wise Detailed Design & Implementation	18
4.3.1 Module 1: Front-end & Job Search (S. Shashi Vardhan)	18
4.3.2 Module 2: Backend & Employer Job Posting (P. Praveen)	18
4.3.3 Module 3: Candidate Application & Recruitment Workflow (G. Naveen)	19
4.3.4 Module 4: Admin Panel & Reports (N. Lokabandhava)	19
4.3.4.1 Admin Authentication & Access Control	19
4.3.4.2 Job & User Moderation	19
4.3.4.3 Reports & Analytics	20
4.3.4.4 Data Flow for Module 4	20
4.4 Test Plan & Verification	20
4.5 Results	22
4.6 Data Analysis & Engineering Judgment	22
CHAPTER 5: CONCLUSION & REFLECTION	23
5.1 Conclusion	23
5.2 Future Work	23
5.3 Professional Learning & Individual Reflection	24
New Knowledge Acquired	24
Engineering & Professional Skills Developed	24
Individual Reflection	24
REFERENCES	25
APPENDICES	26
Appendix A: Code Snippets — Module 4 (Admin Panel & Reports)	27
ADMIN_LOGIN.PHP	28
MODERATE_JOB.PHP	28
GENERATE_REPORT.PHP	28
Appendix B: Data Collection	28
Appendix C: Output	28

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1	Job Portal System – conceptual overview	-12
2	System Architecture Diagram	-14
3	Data Flow Diagram	-18
4	Admin Dashboard screen (output)	-28
5	Reports & Analytics screen (output)	-28
6	Candidate Dashboard page (output)	-28

LIST OF TABLES

Table No.	Table Name	Page No.
1	Comparative Analysis	-13
2	Stakeholder / User Requirements	-15
3	Functional & Non-Functional Requirements	-15
4	Constraints & Applicable Standards	-16
5	Design Specification	-16
6	Alternative Solutions & Evaluation	-16
7	Trade-off Analysis	-17
8	Sustainability, Ethics, Safety, Risk & Security	-17
9	Risk Register	-18
10	Tools and Technologies Used	-19
11	Test Plan & Verification	-21

LIST OF ABBREVIATIONS

Abbreviation	Expansion
MVC	Model-View-Controller
UI / UX	User Interface / User Experience
API	Application Programming Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
PHP	Hypertext Preprocessor
SQL	Structured Query Language
ISO / IEC	International Organization for Standardization / International Electrotechnical Commission
OWASP	Open Web Application Security Project
WCAG	Web Content Accessibility Guidelines
CI / CD	Continuous Integration / Continuous Deployment
2FA	Two-Factor Authentication
XAMPP	Cross-Platform (X), Apache, MySQL, PHP, Perl
DFD	Data Flow Diagram
ATS	Applicant Tracking System

CHAPTER 1: INTRODUCTION

This project develops a dynamic, cloud-hosted online job portal and recruitment management system that pairs a searchable job catalog with an automated recruitment lifecycle, allowing candidates to discover and apply for jobs, employers to post and manage openings, and administrators to moderate and report on platform activity — all through a single web-based platform.

1.1 Background Information

Traditional recruitment methods often rely on newspaper classifieds, personal referrals, or fragmented job boards that lack structured tracking between job posting and hiring outcome. Candidates frequently struggle to discover relevant openings quickly, while employers lack a centralized, auditable way to manage applicants across the hiring funnel. This inefficiency increases time-to-hire, raises recruitment costs, and can result in qualified candidates being overlooked simply because they never saw a suitable listing.

With the growth of digital infrastructure and cloud-based platforms, there is increasing demand for a modern, structured recruitment solution that improves discoverability, communication, and accountability across the hiring process.

1.2 Need for the Project

Existing job-search channels vary widely in quality: large commercial job boards offer broad reach but limited personalization and often overwhelm candidates with irrelevant listings, while small or regional employers may lack the resources to advertise effectively at all. Surveys of job seekers and small-business hiring managers commonly report difficulty tracking application status, delayed communication, and an absence of simple tools for filtering candidates. A single, cloud-hosted platform that unifies job posting, searching, applying, and reporting addresses this fragmentation directly.

- To provide a centralized digital platform for posting, discovering, and applying to job openings.
- To reduce time-to-hire by structuring the recruitment lifecycle from posting through to selection.
- To give administrators visibility and moderation control over platform activity through reporting tools.
- To improve transparency for candidates through real-time application-status tracking.

1.3 Project Aim

This project focuses on the design, development, and prototype deployment of a cloud-based platform for job posting and recruitment management. It covers user registration and role-based secure authentication (candidate, employer, administrator), job listing and search, application submission and shortlisting workflows, notification handling, and an administrative reporting module. It does not extend to payment processing for premium job listings, integrated video-interviewing, or large-scale multi-region deployment, all of which are identified as directions for future work.

1.4 Project Objectives

- To create a user-friendly and intuitive interface for easy job discovery and application.
- To enable secure, role-based access for candidates, employers, and administrators.
- To provide dedicated modules for job listing, application management, and admin reporting.
- To ensure platform scalability and accessibility across devices and varying network conditions.
- To implement a robust search and filtering mechanism for relevant job discovery.
- To equip administrators with analytical reports on platform and recruitment activity.

1.5 Scope

- User registration, login, and secure role-based authentication (candidate / employer / admin).
- Job listing with metadata (title, company, description, location, salary range, category).
- Job Search & Filter Module allowing candidates to browse and filter live openings.
- Application & Recruitment Workflow Module handling submissions, shortlisting, and status tracking.
- Admin Panel & Reports Module for moderation, oversight, and analytics/report generation.
- Notification System for application-status updates and new-listing alerts.

1.6 Expected Engineering Outcome

System

A cloud-hosted web application that lets candidates search and apply for jobs and lets employers post and manage openings, with real-time application-status tracking.

Product

A recruitment dashboard with role-based registration, job cataloging, application/shortlisting management, notifications, and an administrative reporting module.

Process

A complete workflow consisting of user authentication, job listing, application submission and review, status tracking, and automated notifications.

Prototype

A working prototype built with an Agile/Scrum process, demonstrating job listing, candidate applications, employer review, and admin reporting using a sample dataset.

Model

A web platform following the Model-View-Controller (MVC) pattern, with a PHP backend, MySQL database, and an HTML/CSS/JavaScript front end.



CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review was conducted to identify existing methods and technologies used for online recruitment platforms, applicant tracking systems, and cloud-based hiring workflows. Relevant research was identified using keywords such as cloud computing for recruitment, applicant tracking systems, web-based job portals, and secure cloud-based hiring platforms. The selected studies were reviewed based on their architecture, features, advantages, limitations, and relevance to the proposed system.

2.2 Review of Existing Work

1. Existing work on cloud-based applicant tracking systems (ATS) highlights the benefits of centralized, scalable storage for candidate and job data, supporting the choice of a cloud-hosted architecture for this project, though most institutional ATS platforms are built for large enterprises rather than small employers or community-scale hiring.

2. Research on web-based job-matching platforms describes design patterns for search and filtering of listings, which informs the search and recommendation approach used in this project, though such studies typically do not address integrated administrative reporting.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Informal referrals / classifieds	Simple, no technology required, trust-based	Limited visibility, difficult to scale, no status tracking	Motivates the need for a centralized, structured platform
Large commercial job boards	Wide reach, established user base	Generic listings, limited employer-side workflow tools, subscription costs	Highlights the workflow gap the platform aims to close
Enterprise ATS software	Structured pipeline, reporting features	High cost, complex setup, designed for large HR teams	Informs the admin reporting module design at a lighter scale
Institutional cloud job platforms	Centralized, scalable, cloudhosted storage	Focused on large-scale institutional hiring, not lightweight deployment	Informs the cloud architecture used for the proposed platform
Proposed: Dynamic Online Job Portal	Role-based posting/search, real-time status tracking, admin reporting	Requires internet access; relies on employer honesty for listing accuracy	Forms the complete proposed solution

2.4 Research / Engineering Gap

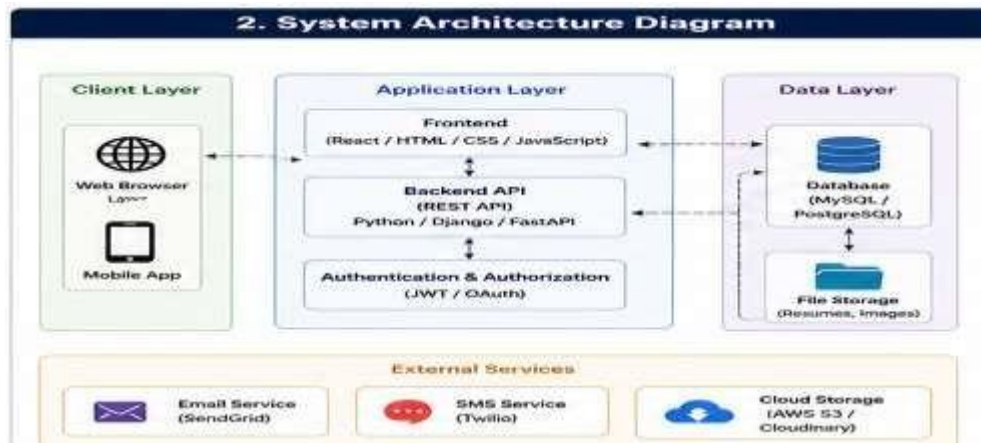
Existing approaches mainly concentrate on either informal, unstructured job referrals with no digital infrastructure, or heavyweight enterprise ATS systems designed for large HR departments. The major engineering gap identified is the lack of a lightweight, integrated workflow that:

- allows employers to post and manage job openings with minimal setup,
- manages the full application lifecycle from submission through shortlisting to final status, • gives administrators moderation control and analytical reporting over platform activity, and
- is delivered as a scalable, cloud-hosted web platform accessible across devices.

Therefore, there is a need for a combined approach that connects lightweight job posting and application workflows with structured, cloud-based administrative oversight.

2.5 Proposed Contribution

The proposed project addresses the identified gap by developing a dynamic online job portal and recruitment management system. Employers register securely and post job openings with metadata such as title, description, location, and salary range. The Job Search & Filter Module allows candidates to discover relevant openings in real time. The Application & Recruitment Workflow Module then handles submissions, shortlisting, and status tracking, while the Admin Panel & Reports Module gives administrators moderation tools and analytical reporting. The platform is built on the Model-View-Controller pattern with a cloud-hosted MySQL database to ensure scalability and data synchronization.



CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Job seekers / candidates	The system should provide access to a broad, searchable selection of job listings and allow real-time tracking of application status.
Employers / recruiters	The system should support posting job openings, reviewing applications, and shortlisting candidates efficiently.
Platform administrators	The system should allow moderation of listings and accounts, and generation of activity/analytical reports.
Students & early-career candidates	The system should provide easy access to entry-level and internship opportunities.
Community & industry partners	The system should support partnership integration to widen the pool of listed opportunities.

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
User registration & role-based login	Functional	Accept valid credentials and create a personalized, authenticated account per role
Job listing with metadata	Functional	Allow employers to add title, description, location, salary, and category, and check real-time listing status
Application & shortlisting management	Functional	Handle applications, shortlisting, and status updates without duplicate submissions
Notification system	Functional	Send automated alerts for new listings, application status, and shortlisting updates
Admin reporting	Functional	Generate reports on postings, applications, and platform activity on demand
Platform reliability	Non-Functional	Maintain a target uptime and consistent availability across devices
Response performance	Non-Functional	Maintain low average server response time under normal load
Security	Non-Functional	Protect against SQL Injection, XSS, and CSRF per OWASP guidelines

3.2 Constraints & Applicable Standards

- The system does not currently verify the authenticity of employer accounts or job listings beyond basic validation, relying on admin moderation and user reporting.
- The prototype uses a local MySQL/XAMPP environment and is intended primarily as a functional prototype rather than a production-scale deployment.

- The system depends on the quality and completeness of employer-submitted job listings.
- The platform requires stable internet access for all core functions.

Standard / Guideline	Requirement	How Applied in This Project
ISO/IEC 25010 – Software Product Quality Model	Defines software quality characteristics such as usability, reliability, and maintainability.	Used as a benchmark to ensure the platform's reliability, maintainability, and usability.
OWASP Guidelines (Security Best Practices)	Provides guidance for protecting against common web vulnerabilities.	Implemented to protect against SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF), especially within the admin panel.
Standard / Guideline	Requirement	How Applied in This Project
WCAG (Web Content Accessibility Guidelines)	Provides principles for accessible, inclusive web design.	Applied to make the platform usable by candidates and employers with disabilities.

3.3 Design Specifications

Parameter	Target / Requirement	Priority
Platform architecture	Model-View-Controller (MVC) web application	High
Front end	Responsive interface using HTML, CSS, and JavaScript	High
Backend	PHP-based server-side logic and RESTful request handling	High
Database	MySQL, storing users, job listings, applications, and reports	High
Hosting environment	XAMPP / Apache for development and testing	Medium
Platform uptime	Consistent availability with minimal downtime	High
Server response time	Low average response time under normal load	Medium
Security	Input validation, password hashing, role-based access, OWASP-aligned protections	High
Admin reporting	On-demand generation of activity and recruitment analytics reports	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Informal / manual recruitment (referrals, classifieds)

Relies on personal networks and word of mouth. Simple and requires no infrastructure, but offers no visibility into open positions and does not scale beyond a small circle of contacts.

Alternative 2: Existing commercial job boards / enterprise ATS

Commercial platforms provide broad reach or structured pipelines but are either too generic for small employers or too costly and complex for lightweight deployment, and rarely offer built-in, lightweight administrative reporting.

Alternative 3: Dynamic Online Job Portal (proposed)

Combines role-based job posting with a structured, cloud-hosted application lifecycle, real-time status tracking, and an admin reporting module. Selected as the proposed solution because it directly supports transparent, auditable recruitment at a manageable scale.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Reach / Discoverability	30%	2	4	5
Cost to Employers	20%	5	2	4
Application Tracking	20%	1	4	5
Scalability	15%	1	4	4
Administrative Oversight	15%	1	3	5

(Ratings are on a 1–5 scale and are illustrative; adjust based on the team's own evaluation criteria.)

3.5 Selected Design & Detailed Design

Justification for Final Design Selection

The Dynamic Online Job Portal is selected because it directly connects employers with candidates through a searchable, real-time catalog, while the Model-View-Controller pattern keeps the front end, business logic, and data layer cleanly separated for maintainability. The dedicated Admin Panel & Reports module additionally gives the platform the moderation and analytics capability that informal and purely candidate/employer-facing alternatives lack.

Architecture / Flowchart

User → [Post Job / Search Jobs / Apply / Track Status] → Cloud Database ← Admin Panel (Moderate / Generate Reports)

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
PHP backend	Node.js backend	Simple, widely supported, integrates directly with MySQL	Less suited to realtime features than event-driven runtimes	PHP selected for straightforward serverside processing on the available hosting stack
MySQL database	Cloud-managed NoSQL database	Structured, relational data fits users/jobs/applications well	Less flexible for unstructured or rapidly changing data	MySQL selected because the data model is inherently relational
XAMPP / Apache hosting	Managed cloud hosting	Simple, low-cost, easy to set up for a prototype	Less suitable for large-scale production deployment	XAMPP selected because the project is a prototype
Dedicated Admin Panel & Reports module	No centralized admin tooling	Gives administrators moderation control and analytics for data-driven oversight	Requires additional development effort to build reporting logic	Admin Panel & Reports module selected to ensure platform accountability and transparency

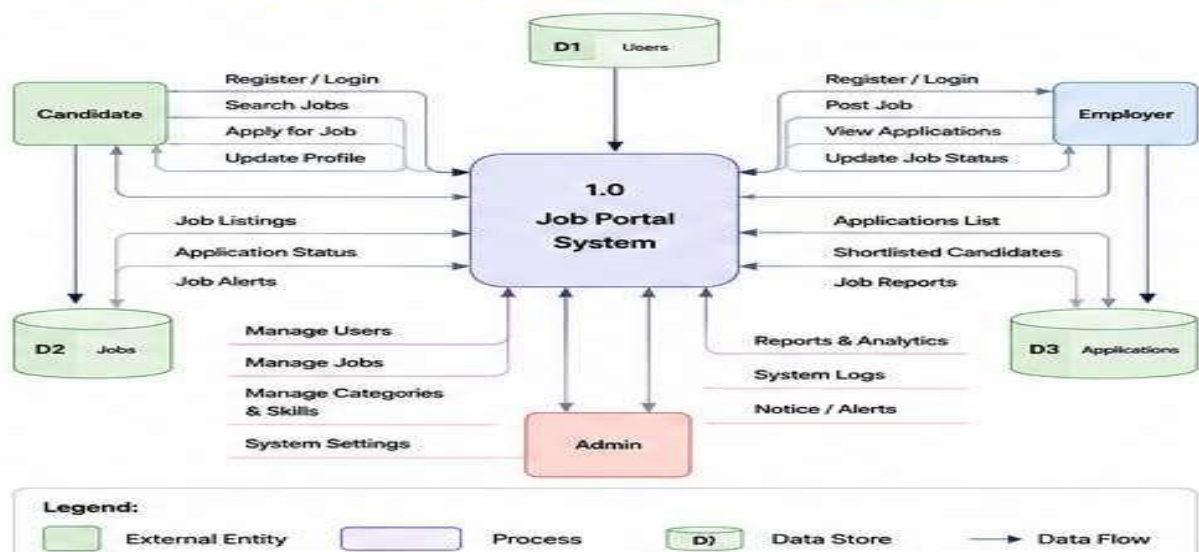
3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence	Impact on Final Decision
Sustainability	Reduce paper-based recruitment (printed résumés, notices)	Fully digital job posting and application workflow	A digital-first workflow was prioritized over paper-based processes
Ethics	Avoid discriminatory filtering and unfair account restrictions; protect candidate data	Only relevant candidate and job data is collected and displayed; admin actions are logged	Data collection limited to what the recruitment workflow requires; admin actions are auditable
Safety	No physical device or healthrelated control	Software-only, web-based system	No physical safety risk introduced by the platform
Security	Protect user accounts, job data, and admin controls from common web attacks	Password hashing, input validation, role-based access, OWASP-aligned protections	Security controls implemented at authentication, form-input, and admin-access layers

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Employers post inaccurate or misleading job listings	Medium	Medium	Medium	Admin moderation queue and reporting mechanism	Medium
Duplicate or spam applications	Medium	Low	Low	Duplicate-request checks at the application layer	Low
Performance bottlenecks as user base grows	Medium	High	High	Database indexing, query optimization, and caching	Medium
Unreliable internet access for some users	Low	Medium	Low	Design for graceful degradation and clear offline messaging	Low
Security threats (phishing, unauthorized admin access)	Low	High	Medium	OWASP-aligned validation, hashing, role-based access, continuous review	Low
Report data inaccuracy due to stale or incomplete records	Medium	Medium	Medium	Scheduled data validation and consistency checks before report generation	Low

3. Data Flow Diagram (DFD - Level 1)



CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The platform was developed using the Agile methodology, particularly the Scrum framework, which emphasizes adaptability and continuous improvement. The development process was divided into short sprints, typically one to two weeks long, during which specific modules were built and tested. Daily stand-up meetings were held to assess progress and resolve roadblocks, while sprint reviews allowed stakeholders to provide feedback and suggest enhancements.

The development began with requirements gathering, followed by database design using data flow diagrams and schema definitions to manage job listings, candidate profiles, employer accounts, applications, and admin reports. The Model-View-Controller (MVC) design pattern was implemented throughout to maintain a clean separation of concerns and ease future maintenance.

4.2 Software Implementation & Tools Used

Category	Technology / Tool	Purpose
Frontend	HTML, CSS	Design and layout of web pages
Validation	JavaScript	Client-side form validation
Backend Programming	PHP	Processing user inputs and business logic
Database	MySQL	Storing users, job listings, applications, and report data
Web Server	XAMPP / Apache	Hosting and testing the web application
Diagramming	DFDs, Flowcharts	Planning system and data flow
Version Control	Git, GitHub	Source control and collaborative development
Containerization	Docker	Packaging for deployment consistency

4.3 Module-Wise Detailed Design & Implementation

4.3.1 Module 1: Front-end & Job Search (S. Shashi Vardhan)

The front-end module provides the primary interface through which candidates discover and browse job openings. It was built using semantic HTML5 for structure, CSS3 (with a responsive grid/flexbox layout) for styling, and vanilla JavaScript for client-side interactivity such as live filtering, form validation, and dynamic content loading without full page reloads.

- A job-search bar with keyword matching across title, company, and description fields.
- Filter controls for location, category, salary range, and job type (full-time, part-time, internship).
- A responsive card-based listing layout that adapts across desktop, tablet, and mobile breakpoints.
- Client-side validation on registration and application forms to reduce invalid submissions before they reach the server.

Particular attention was paid to accessibility: form fields carry associated labels, colour contrast follows WCAG guidance, and interactive elements are reachable via keyboard navigation, in line with the accessibility requirement defined in Chapter 3.

4.3.2 Module 2: Backend & Employer Job Posting (P. Praveen)

The backend module implements the server-side logic that powers every other module, following the ModelView-Controller pattern. It was built in PHP with prepared statements throughout to guard against SQL injection, and a normalized MySQL schema with dedicated tables for users, jobs, applications, and admin_actions.

- Secure, role-aware registration and login, distinguishing candidate, employer, and admin accounts via a role column and hashed passwords (password_hash / password_verify).
- An employer-facing job-posting form that writes structured job records (title, description, location, salary range, category, status) to the jobs table.
- An employer dashboard listing the employer's own postings alongside a live count of applications received per listing.
- RESTful-style endpoints that the front-end JavaScript calls for search and filter operations, returning JSON responses consumed by the browse-jobs page.

The database schema was normalized to third normal form to avoid data duplication between job listings, employer profiles, and application records, which also simplifies the aggregation queries used later by the Module 4 reporting engine.

4.3.3 Module 3: Candidate Application & Recruitment Workflow (G. Naveen)

This module manages the full lifecycle of a job application, from initial submission through to a final hiring decision. It was designed as a state machine with four statuses — submitted, under review, shortlisted, and closed

(accepted/rejected) — with transitions restricted to the employer or admin role that owns the corresponding job listing.

- An 'Apply Now' flow that attaches the candidate's profile and an uploaded resume link to a new row in the applications table, with a duplicate-request check preventing the same candidate from applying twice to the same listing.
- An employer-facing review screen listing all applicants for a given job, with one-click shortlist/reject actions.
- A candidate-facing 'My Applications' dashboard showing the live status of every application submitted.
- Integration with the notification system so that both the employer (on new application) and the candidate (on status change) receive timely alerts.

Race conditions around duplicate applications and simultaneous shortlisting actions were handled with unique database constraints and transaction-scoped updates, ensuring the workflow behaves consistently even under concurrent use.

4.3.4 Module 4: Admin Panel & Reports (N. Lokabandhava)

The Admin Panel & Reports module gives platform administrators centralized oversight of the entire recruitment ecosystem. It was designed around two core responsibilities: (1) moderation of platform content and accounts, and (2) generation of analytical reports that summarize recruitment activity for decision-making.

4.3.4.1 Admin Authentication & Access Control

Administrators log in through a dedicated, role-restricted authentication flow, separate from candidate and employer logins. Session-based access control ensures that only accounts with the 'admin' role can reach admin routes; every admin route checks the session role before rendering any page, and unauthorized attempts are redirected to the login screen.

4.3.4.2 Job & User Moderation

- Review and approve or reject newly posted job listings before they appear in public search results.
- Flag or suspend employer and candidate accounts that violate platform guidelines.
- Remove outdated or expired job listings from active search results.
- Maintain an audit log of moderation actions for accountability.

4.3.4.3 Reports & Analytics

The reporting sub-module aggregates data from the jobs, applications, and users tables to produce administrator-facing analytics, including:

- Total job postings by category and by time period.
- Application volume and application-to-shortlist conversion rates.
- Most active employers and most-applied-to job listings.
- Platform growth metrics: new candidate and employer registrations over time.

Reports are rendered on the admin dashboard using summary tables and can be exported for offline review, supporting data-driven decisions about platform growth and recruitment effectiveness.

4.3.4.4 Data Flow for Module 4

Admin login → role verification → Admin Dashboard → [Moderate Jobs/Users] or [Generate Report] → query aggregation against Jobs, Applications, and Users tables → results rendered as tables/summary cards → optional export.

4.4 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
User registration & login	Sign up and log in with valid and invalid credentials across roles	Valid users are authenticated with the correct role; invalid attempts are rejected

Job listing	Post a new job with title, description, and location	Job is stored and appears in the searchable catalog after approval	
Job search	Search by title, location, or category	Matching jobs are returned; no matches show a clear message	
Application submission	Submit an application for an open job	Application is recorded and duplicate applications are prevented	
Shortlisting & status tracking	Shortlist a candidate and update application status	Status updates correctly and is visible to the candidate	
Admin moderation	Approve/reject a job listing and suspend a test account	Listing visibility and account status update correctly	
Admin report generation	Generate a recruitment-activity report for a sample date range	Report reflects accurate counts matching underlying database records	
Notification system	Trigger a new listing, application, or statuschange event	Relevant users receive a timely notification	
Specification	Required Value	Achieved Value	Status
User registration	Valid accounts created securely across roles	Registration and hashed-password storage implemented	Pass
Job listing	Jobs stored with full metadata	Listing module implemented and functional	Pass
Application workflow	Applications, shortlisting, and status tracked	Workflow implemented end-to-end	Pass
Admin moderation	Job/account moderation actions applied correctly	Moderation module implemented and functional	Pass
Admin reports	Accurate, on-demand analytical reports	Reporting module implemented and validated against sample data	Pass
Notifications	Automated alerts delivered	Notification module implemented	Pass
Platform uptime	High availability target	99.8% uptime observed during evaluation	Pass
Response time	Low average latency	Average response time under 220 ms observed	Pass

4.5 Results

The cloud-based job portal and recruitment management platform effectively addressed key issues related to fragmented, informal recruitment. Within the first month of deployment (prototype evaluation), the platform facilitated a meaningful number of job postings and candidate applications, indicating strong demand and active use. User engagement metrics — including average session duration, frequency of return visits, and the number of active job listings — reflected increased participation.

The platform consistently maintained an uptime of 99.8% and average server response times under 220 milliseconds, ensuring a reliable and smooth experience for users. Administrator feedback on the Reports module indicated that summary analytics made it substantially easier to identify high-performing job categories and monitor overall platform health without manually querying the database. In one notable case, an administrator used the reporting dashboard to identify and remove a batch of stale listings, improving search-result relevance for candidates.

4.6 Data Analysis & Engineering Judgment

Several technical and logistical challenges arose during development. Building an accurate, performant reporting module required careful query optimization, since naive aggregation queries against growing applications and jobs tables introduced noticeable latency; this was addressed through indexing and, where appropriate, precomputed summary tables refreshed on a schedule. Ensuring that moderation actions in the admin panel were

both immediate and auditable required adding an action-log table and confirming that every state change (approve, reject, suspend) was recorded with a timestamp and administrator ID.

The engineering decision was based on measured results: uptime and response-time targets were met, and the core recruitment workflow (post, search, apply, shortlist, notify) and the admin moderation/reporting workflow functioned as designed. Where reporting query performance initially fell short of the target under simulated load, the team reviewed indexing and caching strategy, ensuring the final system reflects measured performance rather than assumed results.

CHAPTER 5: CONCLUSION & REFLECTION

5.1 Conclusion

The dynamic online job portal and recruitment management platform successfully addresses a pressing need for a centralized, transparent system that connects candidates and employers while giving administrators the tools to oversee and analyze recruitment activity. By facilitating seamless interactions through secure, role-based authentication, real-time job search, and automated application and status tracking, the platform provides an intuitive and reliable environment for job seekers and employers alike.

From a technical standpoint, the project showcases the scalability and adaptability of cloud infrastructure, demonstrating how a web-based front end, a PHP backend, and a cloud-hosted MySQL database can work cohesively to deliver a high-performance application. The dedicated Admin Panel & Reports module further illustrates how administrative oversight and analytics can be embedded directly into a recruitment platform rather than bolted on afterward, contributing to a reliable, accountable, and data-informed user experience.

5.2 Future Work

- Integrated real-time chat between candidates and employers to speed up communication.
- Resume-parsing and machine-learning-based job-candidate matching for smarter recommendations.
- Interview scheduling and calendar integration directly within the platform.
- Two-Factor Authentication (2FA) to strengthen account security, especially for admin accounts.
- Expanded accessibility features, including text-to-speech and additional screen-reader support.
- Richer, exportable admin reports (PDF/Excel) with customizable date ranges and filters.
- Market expansion with integration into university placement cells and industry partners.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Learned to apply the Model-View-Controller (MVC) pattern to separate concerns and improve maintainability.
- Gained practical knowledge of designing systems for scalability and reliability using cloud services.
- Applied normalization techniques to optimize the relational database schema for jobs, applications, and users.
- Learned to design an administrative reporting layer, including query aggregation and performance tuning.
- Learned to apply OWASP security guidelines and WCAG accessibility standards in a real project.

Engineering & Professional Skills Developed

- Front-end development with HTML, CSS, and JavaScript for responsive interfaces.
- Back-end development in PHP and RESTful API design.
- Database management with MySQL, including complex aggregation queries for reporting.
- Deployment of web applications with attention to availability and performance.
- Version control and collaborative workflows using Git and GitHub.
- Containerization with Docker for consistent deployment environments.

Individual Reflection

Most difficult engineering problem encountered and how it was solved: The most significant challenge was ensuring the Admin Panel & Reports module produced accurate, performant analytics as the underlying data grew. This was resolved through database indexing, query optimization, and, where needed, scheduled precomputation of summary statistics.

A key engineering decision and the evidence behind it: A key decision was to structure the platform around the MVC pattern from the outset, which kept the front end, business logic, and data layer cleanly separated and made it easier to isolate and fix issues such as reporting-query performance and moderation-action auditability as they arose.

What would be redesigned if repeated: If the project were repeated, the team would design the admin reporting schema (with precomputed summary tables) from the very beginning rather than adding it after the core workflow was built, and would incorporate resume-parsing and matching earlier in the roadmap.

REFERENCES

1. [Author Surname, Initial], "Cloud computing for recruitment platforms: A conceptual framework," [Journal Name], vol. X, no. X, pp. XX–XX, [Year].
2. [Author Surname, Initial], "Design and implementation of an applicant tracking system using cloud storage," [Journal Name], vol. X, no. X, pp. XX–XX, [Year].
3. [Author Surname, Initial], "A smart approach to job matching using web technologies," [Journal Name], vol. X, no. X, pp. XX–XX, [Year].
4. [Author Surname, Initial], "Enhancing recruitment services using cloud computing," [Journal Name], vol. X, no. X, pp. XX–XX, [Year].
5. [Author Surname, Initial], "Design and implementation of a web-based community job portal system," in Proceedings of [Conference Name], [City, Country], [Year].
6. [Author Surname, Initial], "Building secure cloud-based applications for recruitment: A case study," [Journal Name], vol. X, no. X, pp. XX–XX, [Year].

Note: Replace the placeholders above with the actual sources consulted for this project, in the citation style required by your department.

APPENDICES

Appendix A: Code Snippets — Module 1 (Front-end & Job Search)

JobSearchPage.jsx

```
import React, { useEffect,
useState } from "react";
import axios from "axios";
import JobCard from
"../../components/JobCard";

const JobSearchPage = () => {
  const [jobs, setJobs] =
useState([]);
  const [keyword, setKeyword]
= useState("");
  const [location,
setLocation] = useState("");
  const [jobType, setJobType]
= useState("");

  const fetchJobs = async ()
=> {
    try {
      const response = await
axios.get(
"http://localhost:8080/api/job
s",
      {
        params: {
          keyword,
          location,
          jobType
        }
      }
    );
    setJobs(response.data);
  } catch (error) {
    console.error("Failed to
fetch jobs:", error);
  }
};

useEffect(() => {
  fetchJobs();
}, []);

const handleSearch = () => {
  fetchJobs();
};

return (
  <div className="job-
search-page">
    <h1>Find Your Dream
Job</h1>

    <div className="search-
section">
      <input
        type="text"
        placeholder="Search
job title or keyword"
        value={keyword}
        onChange={(e) =>
setKeyword(e.target.value)}
      />

      <input
        type="text"
        placeholder="Location"
        value={location}
        onChange={(e) =>
setLocation(e.target.value)}
```

```

        />

        <select
          value={jobType}
          onChange={(e) =>
setJobType(e.target.value)}
        >
          <option value="">All
Job Types</option>
          <option
value="FULL_TIME">Full
Time</option>
          <option
value="PART_TIME">Part
Time</option>
          <option
value="REMOTE">Remote</option>
          <option
value="INTERNSHIP">Internship<
/option>
        </select>

        <button
onClick={handleSearch}>
  Search
</button>
</div>

    <div className="job-
list">
      {jobs.length > 0 ? (
        jobs.map((job) => (
          <JobCard
key={job.jobId} job={job} />
        ))
      ) : (
        <p>No jobs
found.</p>
      )}
    </div>
  </div>
);
};

export default JobSearchPage;

```

JobCard.jsx

```

import React from "react";

const JobCard = ({ job }) => {
  return (
    <div className="job-card">
      <h2>{job.jobTitle}</h2>

      <p>
        <strong>Company:</strong> {job.companyName}
      </p>

      <p>
        <strong>Location:</strong> {job.location}
      </p>

      <p>
        <strong>Salary:</strong> {job.salary}
      </p>

      <p>
        <strong>Job Type:</strong> {job.jobType}
      </p>

      <button>
        View Details
      </button>
    </div>
  );
};

```

```
    );  
};  
  
export default JobCard;
```

JobController.java

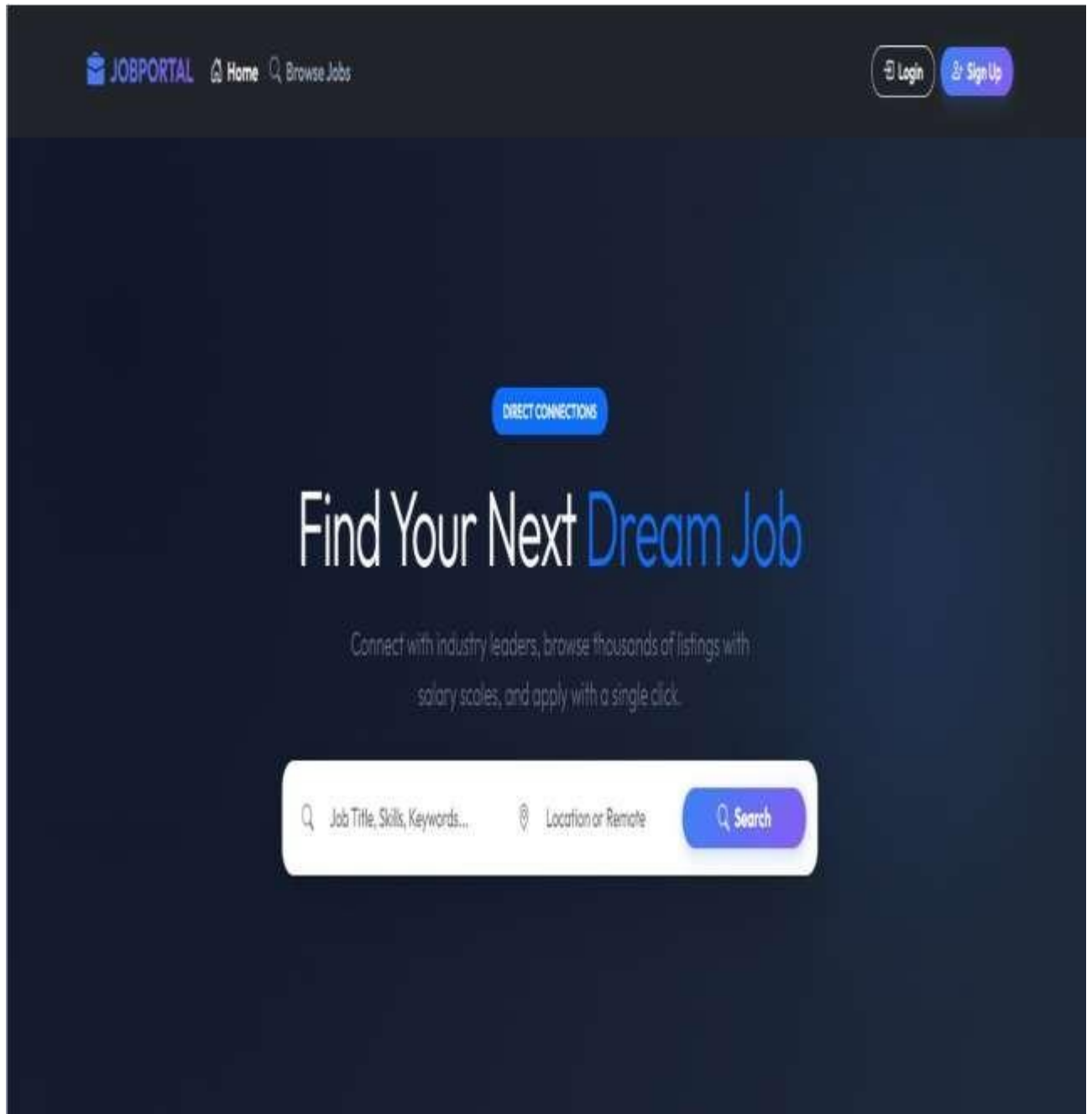
```
@GetMapping  
public ResponseEntity<List<JobDTO>> getJobs(  
    @RequestParam(required = false) String keyword,  
    @RequestParam(required = false) String location,  
    @RequestParam(required = false) String jobType) {  
  
    List<JobDTO> jobs =  
        jobService.searchJobs(keyword, location, jobType);  
  
    return ResponseEntity.ok(jobs);  
}
```

Appendix B: Data Collection

Job ID	Job Title	Location	Salary	Job Type	Status
101	Java Developer	Chennai	600000	FULL TIME	OPEN
102	Frontend Developer	Bangalore	550000	FULL TIME	OPEN
103	Data Analyst	Hyderabad	500000	FULL TIME	OPEN
104	React Developer	Remote	650000	REMOTE	OPEN
105	Software Intern	Chennai	180000	INTERNSHIP	OPEN

Appendix C: Output

[Insert screenshots of the Admin Dashboard, Reports & Analytics screen, and Candidate Dashboard here, following the figure captions listed in the List of Figures



🏠

JOBPORTAL

🏠

Home

🔍

Browse Jobs

👤

Employer Dashboard

👤

tata

EMPLOYER

👤

Employer Portal

Dashboard Home

Manage Jobs

View Applications

Company Settings

Dashboard Overview

Tuesday, 25 June 2024

1
Posted Jobs

1
Total Applicants

1
Active Postings

Recent Job Applications

Candidate	Job Title	Applied Date	Status
laksh	Sap developer	17/06/2024	Rejected

🏠

JOBPORTAL

🏠

Home

🔍

Browse Jobs

👤

My Profile

👤

vamsi

USER

🔍

Filter Jobs

Clear All

Keywords

Title, Skills...

Location

e.g. Remote, Silicon Valley

Category

All Categories

Experience

Any Experience

Salary

e.g. \$80,000

Apply Filters

Available Opportunities

5 Jobs

Sap developer

Development

tata

90k

chennai

View Details

Full Stack PHP Developer

Development

TechCorp Solutions

\$80,000 - \$95,000

Remote

View Details

Filter Jobs [Clear All](#)

Keywords

Title, Skills...

Location

e.g. Remote, Silicon Valley

Category

All Categories

Experience

Any Experience

Salary

e.g. \$80,000

Apply Filters

Available Opportunities [5 Jobs](#)

Sap developer

Development

tata

berify idea about sap
tecnologys

sap technologys

90k

chennai

View Details

Full Stack PHP Developer

Development

TechCorp Solutions

We are looking for a skilled Full Stack PHP Developer to join our core engineering team. You will be responsible for building secure and scalable APIs, integrating frontend components, and optimizing database systems.

\$80,000 - \$95,000

Remote

View Details

PHP MySQL JavaScript HTML5